Provide / Inject en Vue 3

Tutorial: los conceptos básicos y cómo los usamos en el diálogo de autenticación de SpaceTraders

Versión de Vue del proyecto: 3.5 · Octubre 2026

Contenido

1. El problema: pasar datos a muchos niveles
2. La idea de provide / inject
3. Primer ejemplo, paso a paso
4. Reglas básicas
5. Reactividad: proveer un `ref`, no su valor
6. Quién puede cambiar el estado: `readonly()`
7. Claves con `Symbol` e `InjectionKey`
8. El patrón «composable que inyecta»
9. ¿Props, Pinia o provide/inject?
10. Nuestra implementación: `useAuthDialog`
11. Recorrido de un clic
12. Cómo se prueba
13. Resumen y errores frecuentes

1. El problema: pasar datos a muchos niveles

En Vue, la forma normal de comunicar componentes es **props hacia abajo, eventos hacia arriba**. Un padre pasa datos a su hijo con props, y el hijo avisa al padre con `emit`. Funciona muy bien cuando hay uno o dos niveles.

Pero imagina este árbol, donde el estado vive arriba (en `Layout`) y quien lo necesita está muy abajo (`BotonLogin`):

```
Layout              ← aquí vive  const abierto = ref(false)
  └── Cabecera          ← no lo usa, pero tiene que recibirlo y pasarlo
      └── Menu          ← tampoco lo usa, y también lo pasa
          └── BotonLogin  ← el único que de verdad lo necesita
```

Con props, `Cabecera` y `Menu` tendrían que declarar una prop `abierto` y un evento `abrir` solo para reenviarlos. A esto se le llama **prop drilling** («perforar» con props): componentes intermedios que cargan con datos que no les importan. Si mañana cambia el dato, hay que tocar todos los niveles.

2. La idea de provide / inject

Vue ofrece una alternativa: un componente **provee** (`provide`) un valor, y **cualquier descendiente**, esté a la profundidad que esté, lo **inyecta** (`inject`) directamente, sin pasar por los intermedios.

```
Layout        provide('abierto', abierto)   ─────┐
  └── Cabecera    (no sabe nada)                  │  canal directo
      └── Menu    (no sabe nada)                  │
          └── BotonLogin   inject('abierto')   ◄──┘
```

Puedes pensarlo como una *emisora de radio*: el proveedor emite en una frecuencia (la **clave**), y cualquier componente por debajo de él que sintonice esa frecuencia recibe la señal. Los de en medio no tienen que hacer nada.

> Documentación oficial: `vuejs.org/guide/components/provide-inject` (guía) y `vuejs.org/api/composition-api-dependency-injection` (referencia de la API). Vue lo llama *dependency injection*, «inyección de dependencias».

3. Primer ejemplo, paso a paso

El ancestro llama a `provide(clave, valor)` dentro de su `<script setup>`:

`Layout.vue`

```
<script setup lang="ts">
import { provide } from 'vue'

provide('saludo', 'Hola desde el Layout')
</script>

<template>
  <Cabecera />   <!-- no recibe ninguna prop -->
</template>
```

Cualquier descendiente lo recoge con `inject(clave)`:

`BotonLogin.vue (nieto, bisnieto··· da igual)`

```
<script setup lang="ts">
import { inject } from 'vue'

const saludo = inject('saludo')   // 'Hola desde el Layout'
</script>

<template>
  <p>{{ saludo }}</p>
</template>
```

Fíjate en que `Cabecera` no declara nada. El valor «salta» por encima de ella.

4. Reglas básicas

- **Solo hacia abajo.** Un componente puede inyectar lo que proveen sus *ancestros* (padre, abuelo…). Un hermano o un componente de otra rama no lo ve.
- **Se llaman en `setup`, de forma síncrona.** `provide` e `inject` deben ejecutarse directamente en el `<script setup>` (o dentro de un composable llamado desde ahí). No funcionan dentro de un `setTimeout`, después de un `await` ni en un manejador de clic.
- **Gana el más cercano.** Si dos ancestros proveen la misma clave, el descendiente recibe la del ancestro más próximo.

- **Valor por defecto.** Si nadie provee la clave, `inject` devuelve `undefined` y Vue muestra un aviso en consola. Se puede pasar un segundo argumento como valor por defecto: `inject('saludo', 'Hola genérico')`.
- **A nivel de aplicación.** También existe `app.provide(clave, valor)` en `main.ts`; entonces todos los componentes de la app pueden inyectarlo. Así lo hacen por dentro muchos plugins (Vue Router, PrimeVue…).

Ejemplo de «gana el más cercano»:

```
// Abuelo.vue
provide('tema', 'oscuro')

// Padre.vue (hijo de Abuelo)
provide('tema', 'claro')     // "tapa" el valor del abuelo para su rama

// Hijo.vue (hijo de Padre)
inject('tema')               // 'claro'
```

5. Reactividad: proveer un `ref`, no su valor

`provide` no hace nada mágico con la reactividad: entrega exactamente lo que le das. Por eso importa mucho *qué* le das.

```
const contador = ref(0)

provide('contador', contador.value)  // ❌ entrega el número 0, una copia "congelada"
provide('contador', contador)        // ✅ entrega el ref: los cambios llegan al descendiente
```

Si provees el `ref` entero, cuando el proveedor haga `contador.value++`, todos los componentes que lo inyectaron se vuelven a pintar. Si provees `.value`, el descendiente recibe un número suelto y nunca se entera de los cambios.

> Recuerda: en el `<template>` Vue desenvuelve los `ref` de nivel superior automáticamente, así que puedes escribir `{{ contador }}`. En el `<script>` hay que usar `contador.value`.

6. Quién puede cambiar el estado: `readonly()`

Si provees un `ref` tal cual, cualquier descendiente podría hacer `contador.value = 999`. Con muchos componentes tocando el mismo dato, encontrar quién lo cambió se vuelve muy difícil.

La guía oficial recomienda **mantener las mutaciones dentro del proveedor**: el proveedor expone el dato en **solo lectura** y, aparte, **funciones** para cambiarlo.

```
// Proveedor
import { provide, readonly, ref } from 'vue'

const contador = ref(0)
function incrementar() {
  contador.value++
}

provide('contador', {
  valor: readonly(contador),   // se puede leer, no escribir
  incrementar,                 // la única forma de cambiarlo
})

// Descendiente
const { valor, incrementar } = inject('contador')
incrementar()      // ✅ correcto
valor.value = 5    // ❌ Vue lo ignora y avisa en consola
```

`readonly()` devuelve una vista del mismo `ref` que **sigue siendo reactiva** (los cambios hechos desde `incrementar` llegan igual), pero que no se puede modificar desde fuera. Así, toda la lógica de «cómo cambia el estado» vive en un único sitio.

7. Claves con `Symbol` e `InjectionKey`

Hasta ahora las claves eran textos (`'contador'`). Eso tiene dos problemas:

1. **Colisiones:** otra parte de la app (o una librería) podría usar el mismo texto.
2. **Sin tipos:** TypeScript no sabe qué devuelve `inject('contador')` , así que lo trata como `unknown` .

La solución que recomienda Vue es usar un `Symbol` como clave. Un `Symbol` es un valor de JavaScript que es **único**: dos `Symbol('x')` nunca son iguales, así que no puede haber colisiones. Vue además ofrece el tipo `InjectionKey<T>` , que «ata» la clave al tipo del valor:

claves.ts

```
import type { InjectionKey, Ref } from 'vue'

export const ContadorKey: InjectionKey<Ref<number>> = Symbol('Contador')
```

```
// Proveedor
provide(ContadorKey, contador)     // TS comprueba que contador sea Ref<number>
provide(ContadorKey, 'hola')       // ❌ error de tipos

// Descendiente
const c = inject(ContadorKey)       // tipo: Ref<number> | undefined
```

El texto dentro de `Symbol('Contador')` es solo una descripción para depurar (lo verás en Vue Devtools); no se usa para buscar.

8. El patrón «composable que inyecta»

Fíjate en que `inject(ContadorKey)` devuelve `Ref<number> | undefined`: TypeScript nos obliga a contemplar que nadie lo haya provisto. Comprobar eso en cada componente sería repetitivo. La solución habitual es envolver el `inject` en un **composable** (una función `useAlgo()`):

```
export function useContador() {
  const contador = inject(ContadorKey, null)
  if (contador === null) {
    throw new Error('useContador() necesita un proveedor')
  }
  return contador                  // tipo: Ref<number>, ya sin undefined
}
```

Ventajas:

- Los componentes no necesitan saber nada de claves ni de `inject`: solo llaman a `useContador()`.
- Si alguien usa el componente fuera del proveedor, falla **en el acto** con un mensaje claro, en vez de romperse más tarde con un «cannot read properties of undefined».
- Tras el `throw`, TypeScript sabe que el valor existe, así que no hacen falta comprobaciones ni el operador `!`.

9. ¿Props, Pinia o provide/inject?

	Props / emits	**provide / inject**	**Store de Pinia**
Alcance	Padre → hijo directo	Un subárbol: el proveedor y todos sus descendientes	Toda la aplicación
Vida del estado	La del componente	La del componente proveedor: si se desmonta, desaparece	Global, vive mientras dure la app
Visibilidad	Muy explícito: se ve en el template	Implícito: hay que saber quién provee	Explícito por import; visible en Devtools
Cuándo usarlo	1–2 niveles	Estado de una «zona» de la app, a varios niveles	Estado que comparten varias vistas (sesión, agente)

> **Cuidado:** provide/inject hace el flujo de datos menos visible. Leyendo `BotonLogin.vue` no sabes de dónde viene el dato si no buscas quién lo provee. Úsalo cuando el prop drilling sea un problema real, no para todo.

10. Nuestra implementación: `useAuthDialog`

10.1 Qué problema resuelve

La landing pública tiene varios botones que abren el diálogo de acceso: «Iniciar sesión» en la cabecera, «Empezar» en el hero y otro en la sección final (CTA). Todos tienen que abrir **el mismo** diálogo, en la pestaña correcta (login o registro). Y el diálogo, a su vez, necesita saber si está abierto y qué pestaña mostrar.

Este es el árbol real:

```
PublicLayout                          provide(AuthDialogKey, {...})
  ├── AppHeader                         useAuthDialog() → openLogin
  ├── RouterView
  │     └── LandingView
  │         ├── LandingHero              useAuthDialog() → openRegister
  │         └── LandingCta               useAuthDialog() → openRegister
  ├── AppFooter
  └── AuthDialog                        useAuthDialog() → visible, activeTab, close, selectTab
```

Sin provide/inject, `PublicLayout` tendría que pasar el estado por props a `RouterView` y a `LandingView` solo para que llegara al hero y al CTA, y cada botón tendría que emitir un evento que subiera nivel a nivel. `LandingView` se convertiría en un simple «cartero».

10.2 Por qué no un store de Pinia

El comentario de la línea 4 de `useAuthDialog.ts` lo resume: *«PublicLayout lo provee y solo existe bajo ese layout»*. El diálogo de acceso solo tiene sentido en la parte pública de la app. Un store sería global: existiría también en las vistas privadas, donde no hay diálogo de login. Con provide/inject el estado **nace y muere con** `PublicLayout`. Esto encaja con la regla de `doc/architecture.md`: empezar en local y subir a store solo cuando lo necesite más de una vista.

10.3 El archivo, parte por parte

`src/composables/useAuthDialog.ts`

a) El tipo de pestaña

```ts
export type AuthTab = 'login' | 'register'
```

Un *union type*: solo admite esos dos textos. Si alguien escribe `'signup'`, TypeScript da error.

b) El «contrato» de lo que se provee

```ts
export interface AuthDialogContext {
  visible: Readonly<Ref<boolean>>
  activeTab: Readonly<Ref<AuthTab>>
  openLogin: () => void
  openRegister: () => void
  close: () => void
  selectTab: (tab: AuthTab) => void
}
```

Es exactamente el patrón de la sección 6: **dos datos de solo lectura** (`visible`, `activeTab`) y **cuatro funciones** que son la única forma de cambiarlos. El tipo `Readonly<Ref<⋯>>` hace que TypeScript dé error si un componente intenta escribir `visible.value = true`; el `readonly()` del proveedor lo impide además en tiempo de ejecución.

c) La clave

```ts
export const AuthDialogKey: InjectionKey<AuthDialogContext> = Symbol('AuthDialog')
```

La «frecuencia de radio» de la sección 7: un `Symbol` único, tipado con `InjectionKey`, de modo que `provide` exige un `AuthDialogContext` completo e `inject` devuelve ese mismo tipo.

d) El composable

```ts
export function useAuthDialog(): AuthDialogContext {
  const context = inject(AuthDialogKey, null)

  if (context === null) {
    throw new Error(
      'useAuthDialog() necesita un componente antepasado que lo provea (PublicLayout).',
    )
  }

  return context
}
```

El patrón de la sección 8. Dos detalles:

- Se pasa `null` como valor por defecto. Así, si no hay proveedor, Vue no muestra su aviso genérico, sino que lanzamos nuestro propio error con un mensaje que dice exactamente qué falta.
- El tipo de retorno es `AuthDialogContext` (sin `| null`), y los componentes lo usan sin más comprobaciones.

e) Un *type guard* auxiliar

```ts
export function isAuthTab(value: unknown): value is AuthTab {
  return value === 'login' || value === 'register'
}
```

No es parte de provide/inject, pero se usa junto a él. El componente `Tabs` de PrimeVue emite el valor de la pestaña como `string | number`, que es más amplio que `AuthTab`. La firma `value is AuthTab` le dice a

TypeScript: «si esta función devuelve `true`, trata `value` como `AuthTab`». Así podemos pasarlo a `selectTab` sin forzar el tipo.

10.4 El proveedor: `PublicLayout.vue`

`src/components/layout/PublicLayout.vue`

```
const authDialogVisible = ref(false)
const authDialogTab = ref<AuthTab>('login')

function openLogin() {
  authDialogTab.value = 'login'
  authDialogVisible.value = true
}
// openRegister, close y selectTab siguen la misma idea

provide(AuthDialogKey, {
  visible: readonly(authDialogVisible),
  activeTab: readonly(authDialogTab),
  openLogin,
  openRegister,
  close,
  selectTab,
})
```

- El estado real son dos `ref` **privados** del layout. Nadie más tiene acceso de escritura a ellos.
- Se proveen los `ref` enteros (no `.value`), envueltos en `readonly()`: reactivos pero de solo lectura (secciones 5 y 6).
- Las funciones cambian siempre los dos datos de forma coherente: `openRegister` no se limita a abrir, también pone la pestaña correcta. Esa regla vive en un único sitio.
- En el template, `<AuthDialog />` se renderiza **dentro** del layout, junto a `AppHeader` y `RouterView`. Así es descendiente y puede inyectar.

10.5 Los consumidores

Los botones (`AppHeader`, `LandingHero`, `LandingCta`) solo toman lo que necesitan:

```
const { visible, openRegister } = useAuthDialog()
```

Usan `openLogin` u `openRegister` en el `@click`, y `visible` para los atributos de accesibilidad `aria-expanded` y `aria-controls`, que indican a los lectores de pantalla si el botón ha abierto el diálogo `#auth-dialog`.

> **¿Por qué se puede desestructurar sin perder reactividad?** Porque lo que se desestructura es un objeto normal cuyas propiedades son `ref`. `visible` sigue siendo el mismo `ref`. (Lo que sí rompería la reactividad es desestructurar un objeto `reactive()`; aquí no es el caso.)

El diálogo (`AuthDialog.vue`) usa casi todo:

```
const { visible, activeTab, close, selectTab } = useAuthDialog()
```

- `:visible="visible"` controla si el `Dialog` de PrimeVue se muestra.
- `@update:visible="close"` : cuando el usuario cierra (botón X, tecla Esc o clic fuera), PrimeVue emite el evento y llamamos a `close` . El diálogo **no** cambia `visible` por su cuenta; pide al proveedor que lo haga.
- `:value="activeTab"` y `@update:value="handleTabChange"` hacen lo mismo con las pestañas, pasando antes por `isAuthTab` .
- `@switch-to-login="selectTab('login')"` : el formulario de registro puede pedir cambiar a login.

Fíjate en el patrón: es el mismo «datos hacia abajo, eventos hacia arriba» de props/emits, solo que el «arriba» es el proveedor y el «evento» es llamar a una función inyectada.

11. Recorrido de un clic

Qué ocurre cuando el usuario pulsa «Empezar» en el hero:

1. `LandingHero` ejecuta `openRegister()` , la función que inyectó.
2. Esa función es la de `PublicLayout` : pone `authDialogTab.value = 'register'` y `authDialogVisible.value = true` .
3. Vue detecta el cambio en esos `ref` . Como `AuthDialog` , `AppHeader` , `LandingHero` y `LandingCta` leen las versiones `readonly` de esos mismos `ref` , todos se actualizan.
4. El diálogo se abre en la pestaña «Crear cuenta», y los `aria-expanded` de los botones pasan a `true` .
5. El usuario pulsa Esc → PrimeVue emite `update:visible` → `AuthDialog` llama a `close()` → `authDialogVisible.value = false` → todo vuelve al estado inicial.

En ningún momento `LandingView` (el intermedio) ha intervenido.

> **Truco:** en Vue Devtools, selecciona `LandingHero` y verás una sección *injected* con el contexto de `AuthDialog` ; selecciona `PublicLayout` y verás lo que *provee*.

12. Cómo se prueba

- `src/__tests__/useAuthDialog.spec.ts` : monta un componente que llama a `useAuthDialog()` *sin* proveedor y comprueba que lanza el error con el mensaje exacto. También prueba `isAuthTab` con valores válidos e inválidos (`'signup'` , `''` , `'Login'` , números).
- `src/__tests__/AuthDialog.spec.ts` : prueba el diálogo *aislado*, sin `PublicLayout` . En lugar del proveedor real, le inyecta un contexto falso usando la opción `global.provide` de Vue Test Utils. Luego simula clics (en una pestaña, en la X de cierre…) y comprueba que se llamó a la función correcta del contexto (`selectTab` , `close`).
- `src/__tests__/App.spec.ts` : monta la app entera, con el proveedor real, y prueba el recorrido como lo haría un usuario.

La clave para probar un componente que inyecta es que **el test puede hacer de proveedor**:

```
mount(AuthDialog, {
  global: {
    provide: { [AuthDialogKey]: context },   // context: un AuthDialogContext con funciones espía
  },
})
```

Es otra ventaja del diseño: como el componente solo depende del *contrato* (`AuthDialogContext`) y no de `PublicLayout` , se puede sustituir el proveedor por uno falso.

13. Resumen y errores frecuentes

Concepto	En una frase
`provide(clave, valor)`	Un ancestro ofrece un valor a todo su subárbol.
`inject(clave, defecto?)`	Un descendiente lo recoge, sin importar la profundidad.
`ref` provisto	Mantiene la reactividad; `.value` provisto la pierde.
`readonly()`	Los descendientes leen pero no escriben; las mutaciones viven en el proveedor.
`InjectionKey` + `Symbol`	Clave única y tipada.
Composable `useX()`	Oculta el `inject` , quita el `undefined` y falla con un mensaje claro.

Errores frecuentes

- **Proveer** `.value` en lugar del `ref` → los descendientes no se actualizan.
- **Llamar a** `inject` **fuera de** `setup` (en un `onClick` , tras un `await`) → no encuentra nada.
- **Usar el componente fuera del proveedor** (por ejemplo, poner `AppHeader` en un layout privado) → nuestro `useAuthDialog()` lanza el error. Es intencionado: avisa pronto.
- **Mutar el estado desde un descendiente** → evitado gracias a `readonly()` y `Readonly<Ref>` .
- **Abusar de provide/inject** para cosas que un prop resolvería → el flujo de datos se vuelve difícil de seguir.

Fuentes: guía «Provide / Inject» (vuejs.org/guide/components/provide-inject), referencia de la API de inyección de dependencias (vuejs.org/api/composition-api-dependency-injection), guía de TypeScript con la Composition API, sección «Typing Provide / Inject» (vuejs.org/guide/typescript/composition-api), y el código del proyecto en la rama `modal-register-login` .